```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
```

Task 1

```
reviews = [
    "The application provides stable performance during daily operations",
    "There are multiple issues affecting the overall system reliability",
    "User interface is intuitive and easy to understand",
    "The response time is slower than expected under peak load",
    "System updates improved performance and fixed previous issues",
    "The documentation is unclear and lacks sufficient details",
    "The platform maintains consistent functionality across sessions",
    "There are occasional failures during transaction processing",
    "The design is structured and follows standard guidelines",
    "Performance degradation is observed during extended usage"
]
selected_review = reviews[0]

print("Selected Review:\n")
print(selected_review)
```

```
Selected Review:

The application provides stable performance during daily operations
```

```
words = selected_review.split()[:100]
review = " ".join(words)
```

Task 2

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(reviews)
```

```
sequence = tokenizer.texts_to_sequences([review])
```

Task 3

```
max_length = 100

padded_sequence = pad_sequences(
    sequence,
    maxlen=max_length,
    padding='post'
)

print("\nPadded Sequence Shape:", padded_sequence.shape)
```

```
Padded Sequence Shape: (1, 100)
```

Task 4

```
vocab_size = len(tokenizer.word_index) + 1
embedding_dim = 16

embedding_layer = tf.keras.layers.Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim
)

embedded = embedding_layer(padded_sequence)

print("Embedding Output Shape:", embedded.shape)
```

```
Embedding Output Shape: (1, 100, 16)
```

```
attention_dense = tf.keras.layers.Dense(1)
```

```
attention_scores = attention_dense(embedded)

# Shape: (1,100,1)
attention_scores = tf.squeeze(attention_scores, axis=-1)
attention_weights = tf.nn.softmax(attention_scores, axis=1)
```

```
print("\nWord\t\tAttention Weight")
print("-"*40)

attention_values = attention_weights.numpy()[0]

for i, word in enumerate(words):
    print(f"{word:15s} {attention_values[i]:.6f}")
```

```
Word            Attention Weight
----------------------------------------
The             0.010267
application     0.009893
provides        0.010504
stable          0.010027
performance     0.011022
during          0.009618
daily           0.010759
operations      0.010237
```